BABU BANARASI DAS UNIVERSITY

LUCKNOW

Session : 2025-2026

SCHOOL OF COMPUTER APPLICATIONS

CASE STUDY

ON

Student Performance Analysis using Hadoop

Submitted To:-

Mr. Vikas (IBM)

Submitted By:-

Group No. – 7(G)

Section – BCADS24

PROJECT

Student Dataset Mapreduce Practical

Architecture:

- The architecture of this project follows a standard Big Data pipeline using components of the Hadoop ecosystem to process and analyze student performance data.

Data Flow:

Student Dataset (CSV)
↓
HDFS (Storage)
↓
MapReduce Processing
↓
Processed Data (HDFS)
↓
Hive Tables
↓
Analytical Queries
↓ Insights
& Results

Explanation:

- Student dataset is uploaded to **HDFS**
- Data is processed using **MapReduce**
- Operations like **filtering, grouping, and aggregation** are performed
- Processed data is stored back in **HDFS**
- Data is loaded into **Hive tables** • **Hive queries** are used for analysis
- Final **insights are generated.**

Tools Used:

- Hadoop (HDFS) – for storing large dataset in distributed manner
- MapReduce – for processing data (filtering, grouping, aggregation)
- Hive – for querying and analyzing data using SQL-like language
- Python – for writing Mapper and Reducer programs
- Linux (Cloudera VM) – environment to run Hadoop command

- ChatGPT (Generative AI tool)

○ Technologies:

- Python (for mapper and reducer)
- CSV dataset
- Hadoop Ecosystem

○ Use of Generative AI in the Project:

- Generative AI is used to assist in coding, querying, analysis, and documentation of the project.

○ Areas Where AI Was Used

- **Code Generation** – for writing Mapper and Reducer programs
- **Query Assistance** – for creating Hive queries
- **Concept Understanding** – for explaining Hadoop, MapReduce, and Hive
- **Data Analysis Support** – for interpreting results and identifying patterns
- **Debugging Help** – for fixing errors in code and commands

○ Example Prompts Used

- Generate Python code for Mapper and Reducer for filtering data.

○ Benefits of Using AI:

- Saves Time
- Improves Productivity
- Easy Understanding
- Error Reduction
- Guidance Support

○ <u>Validation:</u>

- All AI-generated code was manually tested and verified before execution.

○ Dataset Description

- The dataset is a CSV file containing student records, where each row represents a student and each column represents an attribute. It is used for analyzing student data using Hadoop.

○ Dataset Structure

- **Student_ID –** Unique identifier for each student
- **Name –** Name of the student
- **Gender –** Gender of the student
- **Attendance –** Attendance percentage of the student
- **Study Hour –** Number of hours spent studying
- **Previous Grade –** Grade obtained previously
- **Extracurricular Activity –** Participation in activities
- **Parental Factors –** Information related to parents

○ Dataset Source

- The dataset used in this project is a student performance dataset obtained from **Kaggle**, an online platform for datasets and data science projects. It is used for academic purposes to perform analysis using Hadoop tools.

○ Purpose of Dataset

- To **analyze student data**
- To understand **patterns in student performance**
- To perform **filtering, grouping, and aggregation**
- To extract **meaningful insights**
- To demonstrate **use of Hadoop tools for data analysis**

○ Conclusion

- This project demonstrates the use of Hadoop technologies to efficiently process and analyze student data. By using HDFS for storage, MapReduce for data processing, and Hive for querying, large datasets can be handled effectively. The project helps in identifying patterns and extracting useful insights from the data, showing the importance of Big Data tools in data analysis.

HDFS Implementation

Step-1: Upload a file **"Student_performance.csv"** in the Cloudera terminal and then create a folder in terminal with name **"Student_performance_project"**

```
[cloudera@quickstart ~]$ sudo VBoxControl sharedfolder list
Oracle VM VirtualBox Guest Additions Command Line Management Interface Version 5
.0.14
(C) 2008-2016 Oracle Corporation
All rights reserved.

Shared Folder mappings (1):

01 - BD dataset
```

```
[cloudera@quickstart shared]$ ls
student_performance.csv
[cloudera@quickstart shared]$ cp student_performance.csv /home/cloudera/
cp: overwrite `/home/cloudera/student_performance.csv'? y
[cloudera@quickstart shared]$ cp student_performance.csv /home/cloudera/
cp: overwrite `/home/cloudera/student_performance.csv'? y^C
[cloudera@quickstart shared]$ cd /home/cloudera
[cloudera@quickstart ~]$ ls
biddata_demo                lib              student_performance.csv
bigdata_demo                -mapper          student.txt
bigdata_demo1               mapper.py        Templates
cloudera-manager            mapreduce_d      Videos
cm_api.py                   mapreduce_de     vidhi
Desktop                     mapreduce_demo   vidhi_file.txt
Documents                   mapreduce_demo1  Vidhi_folder
Downloads                   mapreduce_demo2  vidhi_local.txt
eclipse                     Music            vidhi_retrieved.txt
enterprise-deployment.json  parcels          Vidhi.txt
express-deployment.json     Pictures         word.txt
healthcare_dataset.csv      Public           workspace
kerberos                    reducer.py
[cloudera@quickstart ~]$ hdfs dfs -mkdir /student_performance_project
[cloudera@quickstart ~]$ hdfs dfs -put student_performance.csv  /student_performance_project
```

MapReduce Implementation

. Write at least ONE MapReduce program

Mapper.py :

```
PS C:\student_mapreduce> Write-Host "===== MAPREDUCE PROGRAM - mapper.py =====" ; type mapper.py
===== MAPREDUCE PROGRAM - mapper.py =====
import sys
import csv

with open("student_performance.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        gender = row["Gender"].strip()
        if gender:
            print(gender + "\t" + "1")
```

reducer.py

```
PS C:\student_mapreduce> Write-Host "===== MAPREDUCE PROGRAM - reducer.py =====" ; type reducer.py
===== MAPREDUCE PROGRAM - reducer.py =====
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    line = line.strip()
    word, count = line.split("\t", 1)
    count = int(count)

    if current_word == word:
        current_count += count
    else:
        if current_word:
            print(current_word + "\t" + str(current_count))
        current_word = word
        current_count = count

if current_word:
    print(current_word + "\t" + str(current_count))
```

mapper_avg.py

```
PS C:\student_mapreduce> Write-Host "===== AGGREGATION PROGRAM - mapper_avg.py =====" ; type mapper_avg.py
===== AGGREGATION PROGRAM - mapper_avg.py =====
import csv

with open("student_performance.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        gender = row["Gender"].strip()
        grade = row["FinalGrade"].strip()
        if gender and grade:
            print(gender + "\t" + grade)
PS C:\student_mapreduce>
```

- **Perform meaningful transformation such as:**
  - **Counting**

```
PS C:\student_mapreduce> type students.txt | python mapper.py
Male    1
Female  1
Male    1
Male    1
Female  1
Male    1
Female  1
Female  1
Male    1
Female  1
PS C:\student_mapreduce> type students.txt | python mapper.py | sort | python reducer.py
Female  5
Male    5
PS C:\student_mapreduce>
```

  - **Filtering**

```
PS C:\student_mapreduce> python mapper.py | sort | python reducer.py
Female  451
Male    501
PS C:\student_mapreduce> python mapper_grade.py | sort | python reducer.py
A_Grade 193
B_Grade 474
C_Grade 293
PS C:\student_mapreduce>
```

  - **Aggregation**

```
PS C:\student_mapreduce> python mapper_avg.py | sort | python reducer_avg.py
Female  80.48
Male    79.83
PS C:\student_mapreduce>
```

Hive Implementation

Step-01:- Open Hive

- hive

```
[cloudera@quickstart ~]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive>
```

Step-02:- Create table on hive.

- CREATE TABLE student_performance (
- gender STRING,
- marks FLOAT
-)
- ROW FORMAT DELIMITED
- FIELDS TERMINATED BY '\t'
- STORED AS TEXTFILE;

```
hive> CREATE TABLE student_performance (
    >     gender STRING,
    >     marks FLOAT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY '\t'
    > STORED AS TEXTFILE;
OK
Time taken: 0.44 seconds
```

Step 3: Show tables.

- SHOW TABLES;

```
hive> SHOW TABLES;
OK
student_performance
Time taken: 0.112 seconds, Fetched: 1 row(s)
hive> DESCRIBE student_performance;
OK
gender                  string
marks                   float
Time taken: 0.178 seconds, Fetched: 2 row(s)
```

Step-04:- Load data on the table in student_performance.

- LOAD DATA INPATH '/user/hive/warehouse/student_performance /student_performance.csv'
- INTO TABLE student_performance;

```
hive> LOAD DATA INPATH'/user/hive/warehouse/student_performance/student_performance.csv'
    > INTO TABLE student_performance;
Loading data to table default.student_performance
Table default.student_performance stats: [numFiles=1, totalSize=69015]
OK
Time taken: 0.627 seconds
hive>
```

cloudera@quickstart:~

Step-05:- View Data

- SELECT * FROM student_performance LIMIT 10;

```
hive> SELECT * FROM student_performance LIMIT 10;
OK
StudentID,Name,Gender,AttendanceRate,StudyHoursPerWeek,PreviousGrade,ExtracurricularActivities,ParentalSupport,FinalG
rade,Study Hours,Attendance (%),Online Classes Taken    NULL
1.0,John,Male,85.0,15.0,78.0,1.0,High,80.0,4.8,59.0,False       NULL
2.0,Sarah,Female,90.0,20.0,85.0,2.0,Medium,87.0,2.2,70.0,True   NULL
3.0,Alex,Male,78.0,10.0,65.0,0.0,Low,68.0,4.6,92.0,False        NULL
4.0,Michael,Male,92.0,25.0,90.0,3.0,High,92.0,2.9,96.0,False    NULL
5.0,Emma,Female,,18.0,82.0,2.0,Medium,85.0,4.1,97.0,True        NULL
6.0,Olivia,Female,95.0,30.0,88.0,1.0,High,,2.8,97.0,False       NULL
7.0,Daniel,Male,70.0,8.0,60.0,0.0,Low,62.0,4.5,96.0,False       NULL
8.0,Sophia,Female,,17.0,77.0,1.0,Medium,78.0,1.0,70.0,True      NULL
9.0,James,Male,82.0,12.0,70.0,2.0,Low,72.0,3.6,50.0,False       NULL
Time taken: 0.327 seconds, Fetched: 10 row(s)
hive>
```

Step-06:- Average marks by gender.

- SELECT gender, AVG(marks) AS avg_marks
- FROM student_performance
- GROUP BY gender;

```
hive> SELECT gender, AVG(marks) AS avg_marks
    > FROM student_performance
    > GROUP BY gender;
```

Browsing HDFS - Mozil... | cloudera@quickstart:~

```
9617.0,Emily Torres,,85.0,,60.0,0.0,High,78.0,3.9,66.0,False    NULL
9622.0,Christian Knight,Female,90.0,,60.0,3.0,High,90.0,1.5,58.0,       NULL
9633.0,Alexander Thompson,Female,90.0,20.0,65.0,0.0,High,92.0,2.7,94.0,False    NULL
9633.0,Dawn Taylor,Female,85.0,25.0,60.0,3.0,High,80.0,1.0,63.0,True    NULL
9647.0,Todd Brooks,Male,70.0,17.0,90.0,2.0,Low,88.0,3.3,94.0,False      NULL
9691.0,Haley Robertson,Female,82.0,15.0,85.0,1.0,Medium,92.0,4.5,89.0,True      NULL
9706.0,Tiffany Moore,Male,90.0,17.0,65.0,3.0,Low,80.0,4.5,99.0,False    NULL
9714.0,Erica Thomas,Female,85.0,22.0,88.0,2.0,High,78.0,0.4,75.0,True   NULL
9723.0,Emily Martin,,78.0,17.0,70.0,,Low,62.0,2.0,67.0,True     NULL
9741.0,Erica Miller,Male,85.0,10.0,90.0,0.0,Low,85.0,4.5,98.0,False     NULL
9743.0,Amy Lawson,Male,92.0,22.0,60.0,1.0,High,68.0,1.2,,False  NULL
9743.0,Mr. Clinton Camacho,Female,90.0,15.0,60.0,1.0,High,78.0,4.6,,True        NULL
9747.0,Angela Perez,Female,95.0,30.0,70.0,1.0,Medium,62.0,3.8,77.0,False        NULL
9747.0,James Bennett,Female,70.0,,70.0,2.0,High,62.0,0.7,92.0,True      NULL
9751.0,Rebecca Smith,Male,70.0,22.0,86.0,1.0,High,62.0,0.7,79.0,True    NULL
9779.0,Vincent Johnson,Male,92.0,22.0,,1.0,High,68.0,1.4,51.0,False     NULL
9791.0,Brittany Andrews,Male,91.0,25.0,85.0,1.0,Low,68.0,2.4,51.0,False NULL
9797.0,Donna Taylor,Female,95.0,20.0,82.0,,High,78.0,0.0,96.0,False     NULL
9804.0,Henry Johnson,Male,78.0,15.0,77.0,2.0,High,87.0,4.5,96.0,False   NULL
9806.0,Destiny Todd,Female,82.0,17.0,78.0,1.0,Medium,88.0,1.4,60.0,True NULL
9810.0,Morgan Gilbert,Female,90.0,25.0,77.0,0.0,Low,87.0,5.0,54.0,True  NULL
9814.0,Felicia Howard,Male,90.0,25.0,90.0,0.0,High,92.0,2.7,94.0,False  NULL
9837.0,Jane Griffith,Female,85.0,12.0,65.0,2.0,High,62.0,1.3,87.0,False NULL
9837.0,Susan Brown,Male,91.0,30.0,77.0,1.0,High,90.0,4.4,69.0,True      NULL
9841.0,Julie Garza,Male,85.0,22.0,90.0,3.0,Medium,90.0,4.5,,False       NULL
9850.0,Christopher Baxter,Female,78.0,,78.0,0.0,High,90.0,3.9,65.0,False        NULL
9857.0,Taylor Burns,Female,92.0,22.0,85.0,1.0,Low,85.0,0.5,54.0,True    NULL
9877.0,Crystal Chandler,Male,91.0,12.0,90.0,1.0,Low,80.0,0.6,84.0,True  NULL
9880.0,Darlene Fletcher,Male,85.0,12.0,90.0,0.0,High,92.0,1.5,,False    NULL
9882.0,Nathan Rodgers,Male,82.0,15.0,85.0,1.0,High,62.0,0.7,90.0,True   NULL
9892.0,David Gonzalez,Female,92.0,22.0,65.0,2.0,Medium,,3.7,67.0,True   NULL
9906.0,Sean Lyons,Male,,15.0,85.0,2.0,Medium,,2.6,76.0,True     NULL
9907.0,Rebecca Mendoza DDS,Male,95.0,12.0,86.0,2.0,High,92.0,3.3,72.0,True      NULL
9908.0,Jodi Taylor,Female,95.0,18.0,82.0,1.0,High,72.0,2.9,59.0,True    NULL
9937.0,John White,Female,95.0,20.0,85.0,1.0,Low,85.0,1.5,88.0,False     NULL
9948.0,Taylor Moyer,,88.0,17.0,82.0,3.0,,68.0,4.0,96.0,False    NULL
9971.0,David Brown,Male,85.0,12.0,65.0,3.0,Medium,92.0,3.3,97.0,False   NULL
9972.0,Jacob Thompson,Male,85.0,20.0,78.0,0.0,High,72.0,0.5,,False      NULL
9975.0,Jason Castro,Female,82.0,,90.0,3.0,Low,85.0,3.4,93.0,True        NULL
9976.0,Jason Ramirez,Female,78.0,17.0,77.0,2.0,High,85.0,3.8,83.0,False NULL
9978.0,Paul Poole,Female,90.0,10.0,88.0,3.0,High,85.0,2.3,60.0,True     NULL
9991.0,Terri Potter,Female,91.0,10.0,78.0,0.0,Low,72.0,0.2,60.0,True    NULL
9998.0,Jeffrey Howard,Male,88.0,17.0,77.0,2.0,High,88.0,,95.0,True      NULL
StudentID,Name,Gender,AttendanceRate,StudyHoursPerWeek,PreviousGrade,ExtracurricularActivities,ParentalSupport,FinalG
rade,Study Hours,Attendance (%),Online Classes Taken    NULL
Time taken: 15.509 seconds, Fetched: 1001 row(s)
hive>
```

Step-07:- Count of students.

- SELECT gender, COUNT(*) AS total_students • FROM student_performance • GROUP BY gender.

```
hive> SELECT gender, COUNT(*) AS total_students
    > FROM student_performance
    > GROUP BY gender;
Query ID = cloudera_20260422015555_f006d3ae-6ce5-4ab7-a6b1-d4f48a076d3d
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776299861636_0005, Tracking URL = http://quickstart.cloudera:8088/proxy/application_17762998
_0005/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1776299861636_0005
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-22 01:55:50,428 Stage-1 map = 0%,  reduce = 0%
```

Step-08:- Pass / Fail (assuming pass ≥ 40).

- SELECT
- CASE
- WHEN marks >= 40 THEN 'Pass'
- ELSE 'Fail'
- END AS result,
- COUNT(*) AS count
- FROM student_performance
- GROUP BY
- CASE
- WHEN marks >= 40 THEN 'Pass'
- ELSE 'Fail'
- END;

```
hive> SELECT
    >   CASE
    >     WHEN marks >= 40 THEN 'Pass'
    >     ELSE 'Fail'
    >   END AS result,
    >   COUNT(*) AS count
    > FROM student_performance
    > GROUP BY
    >   CASE
    >     WHEN marks >= 40 THEN 'Pass'
    >     ELSE 'Fail'
    >   END;
Query ID = cloudera_20260422015656_f39e7a3a-10e2-4530-84a1-13c91142dd31
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776299861636_0006, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1776299861636
_0006/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1776299861636_0006
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-22 01:56:39,337 Stage-1 map = 0%,  reduce = 0%
2026-04-22 01:56:45,815 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 1.2 sec
2026-04-22 01:56:50,955 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 1.92 sec
MapReduce Total cumulative CPU time: 1 seconds 920 msec
Ended Job = job_1776299861636_0006
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 1.92 sec   HDFS Read: 77754 HDFS Write: 10 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 920 msec
OK
Fail    1001
Time taken: 17.467 seconds, Fetched: 1 row(s)
hive>
```

Step-09:- Top scores.

- SELECT *
- FROM student_performance
- ORDER BY marks DESC
- LIMIT 5;

```
hive> SELECT *
    > FROM student_performance
    > ORDER BY marks DESC
    > LIMIT 5;
Query ID = cloudera_20260422015757_b1b41e57-252e-42f6-8e68-7bf42aee2f6e
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776299861636_0007, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1776299861636
_0007/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1776299861636_0007
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-22 01:57:32,684 Stage-1 map = 0%,  reduce = 0%
2026-04-22 01:57:38,026 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 1.43 sec
2026-04-22 01:57:43,212 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 2.16 sec
MapReduce Total cumulative CPU time: 2 seconds 160 msec
Ended Job = job_1776299861636_0007
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 2.16 sec   HDFS Read: 76101 HDFS Write: 330 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 160 msec
OK
8021.0,Maria Walls,Female,88.0,10.0,90.0,1.0,Medium,,2.4,94.0,True      NULL
7636.0,Joseph Sherman,Male,88.0,17.0,60.0,2.0,High,85.0,0.9,53.0,True   NULL
1886.0,,Male,85.0,8.0,82.0,2.0,Low,68.0,0.4,54.0,False  NULL
4497.0,Amy Stout,Female,91.0,,86.0,0.0,High,90.0,3.9,80.0,True  NULL
,Kenneth Murray,Male,85.0,20.0,,1.0,High,72.0,0.8,80.0,True     NULL
Time taken: 17.83 seconds, Fetched: 5 row(s)
hive>
```

Use of Generative AI

- **Use AI to generate:**
  - MapReduce code:

Write a Python MapReduce program to count male and female students from a student performance dataset (student_performance.csv)

Python MapReduce Program

Python Run

```python
import csv
from collections import defaultdict

# Step 1: Mapper
def mapper(file_path):
    mapped_data = []

    with open(file_path, 'r') as file:
        reader = csv.DictReader(file)
        for row in reader:
            gender = row['Gender'].strip()
            mapped_data.append((gender, 1))  # Emit (Gender, 1)

    return mapped_data


# Step 2: Shuffle & Sort (Group by key)
def shuffle_sort(mapped_data):
    grouped_data = defaultdict(list)

    for key, value in mapped_data:
        grouped_data[key].append(value)

    return grouped_data


# Step 3: Reducer
def reducer(grouped_data):
    result = {}

    for gender, values in grouped_data.items():
        result[gender] = sum(values)

    return result


# Main Function
if __name__ == "__main__":
    file_path = "student_performance.csv"

    mapped = mapper(file_path)
    grouped = shuffle_sort(mapped)
    reduced = reducer(grouped)

    print("Student Count by Gender:")
    for gender, count in reduced.items():
        print(f"{gender}: {count}")
```

○

Hive queries:

Write Hive queries to analyze student performance dataset with columns: gender, FinalGrade. Include:

1. Average marks by gender

```sql
SELECT
    gender,
    AVG(FinalGrade) AS avg_marks
FROM student_performance
GROUP BY gender;
```

2. Count of students by gender

```sql
SELECT
    gender,
    COUNT(*) AS total_students
FROM student_performance
GROUP BY gender;
```

3. Top 5 students by marks

```sql
SELECT *
FROM student_performance
ORDER BY FinalGrade DESC
LIMIT 5;
```

4. Pass/Fail count (pass >= 60)

```sql
SELECT
    CASE
        WHEN FinalGrade >= 60 THEN 'Pass'
        ELSE 'Fail'
    END AS status,
    COUNT(*) AS count
FROM student_performance
GROUP BY
    CASE
        WHEN FinalGrade >= 60 THEN 'Pass'
        ELSE 'Fail'
    END;
```

○

Refine prompts to improve output Write a Python MapReduce program using mapper.py and reducer.py to calculate average FinalGrade grouped by Gender from student_performance.csv dataset with 1000 student records

1. mapper.py

Python Run

```python
#!/usr/bin/env python3
import sys
import csv

reader = csv.reader(sys.stdin)

# Skip header (optional, safer approach)
header = next(reader, None)

for row in reader:
    try:
        gender = row[0].strip()
        grade = float(row[1])

        # Emit: Gender \t grade
        print(f"{gender}\t{grade}")

    except:
        continue
```

○

2. reducer.py

Python Run

```python
#!/usr/bin/env python3
import sys

current_gender = None
total = 0
count = 0

for line in sys.stdin:
    try:
        gender, grade = line.strip().split("\t")
        grade = float(grade)

        if current_gender == gender:
            total += grade
            count += 1
        else:
            if current_gender:
                avg = total / count
                print(f"{current_gender}\t{avg:.2f}")

            current_gender = gender
            total = grade
            count = 1

    except:
        continue

# Output last key
if current_gender:
    avg = total / count
    print(f"{current_gender}\t{avg:.2f}")
```

Debug AI-generated errors

I am getting this error in my MapReduce reducer.py:

"ValueError: not enough values to unpack"

My reducer code is: for line in sys.stdin:

word, count = line.split("\t", 1) count = int(count) How to fix this error?

○

✅ Fix 1 (Best & Safe Way)

Add validation before splitting:

Python Run

```python
import sys

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue  # skip empty lines

    parts = line.split("\t")

    if len(parts) != 2:
        continue  # skip invalid lines

    word, count = parts

    try:
        count = int(count)
    except ValueError:
        continue  # skip invalid numbers

    # process normally
    print(word, count)
```

✅ Fix 2 (Minimal Change)

Just wrap in try-except:

Python Run

```python
for line in sys.stdin:
    try:
        word, count = line.strip().split("\t", 1)
        count = int(count)
        print(word, count)
    except:
        continue
```